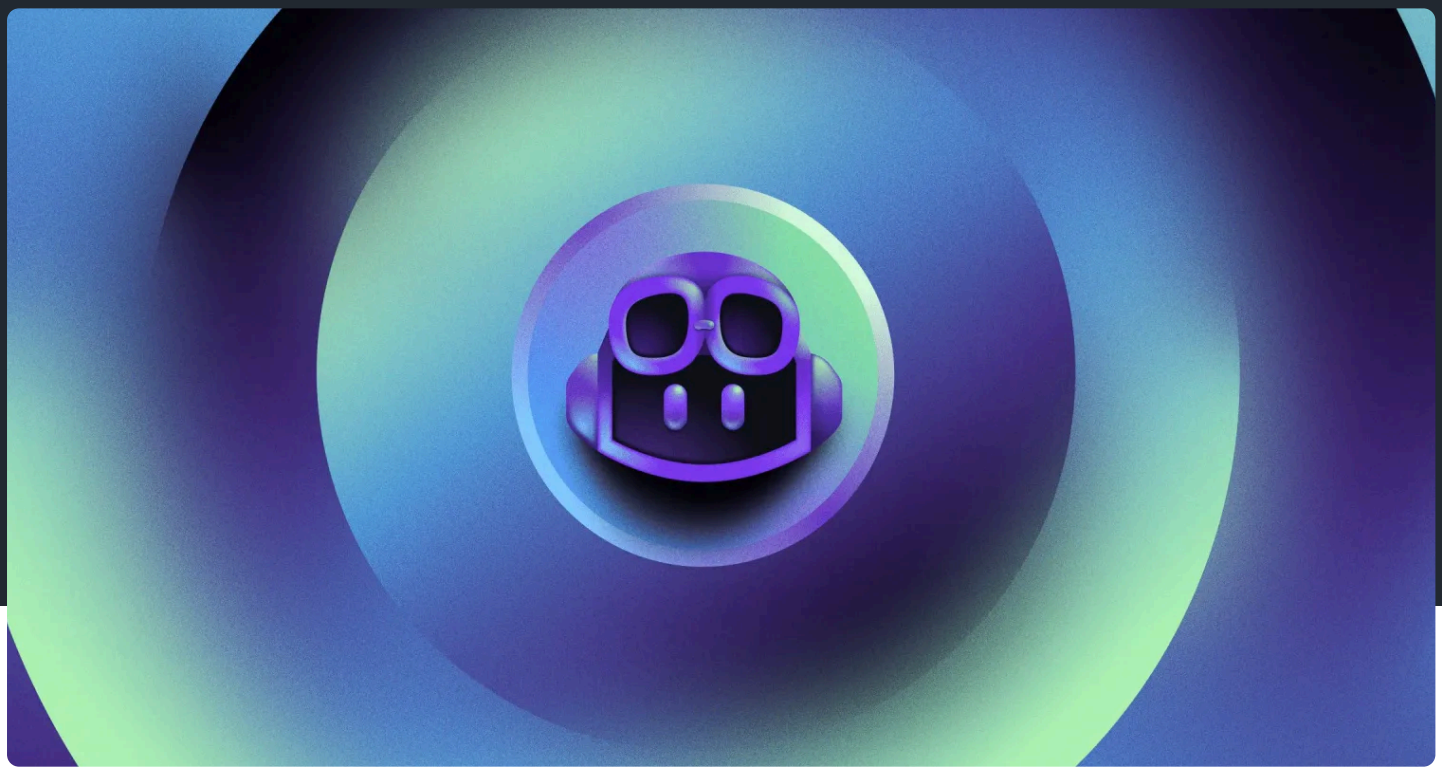
How GitHub Copilot is getting better at understanding your code

With a new Fill-in-the-Middle paradigm, GitHub engineers improved the way GitHub Copilot contextualizes your code. By continuing to develop and test advanced retrieval algorithms, they're working on making our AI tool even more advanced.



Johan Rosenkilde · @johanrosenkilde

May 17, 2023 | Updated February 7, 2024 | 10 minutes

Share:

programmer, GitHub's machine learning experts have been busy researching, developing, and testing new capabilities—and many are focused on improving the AI pair programmer's contextual understanding. That's because good communication is key to pair programming, and inferring context is critical to making good communication happen.

To pull back the curtain, we asked GitHub's researchers and engineers about the work they're doing to help GitHub Copilot improve its contextual understanding. Here's what we discovered.

From OpenAI's Codex model to GitHub Copilot

When OpenAI released GPT-3 in June 2020, GitHub knew developers would benefit from a product that leveraged the model specifically for coding. So, we gave input to OpenAI as it built Codex, a descendant of GPT-3 and the LLM that would power GitHub Copilot. The pair programmer launched as a technical preview in June 2021 and became generally available in June 2022 as the world's first at-scale generative AI coding tool.

To ensure that the model has the best information to make the best predictions with speed, GitHub's machine learning (ML) researchers have done a lot of work called prompt engineering (which we'll explain in more detail below) so that the model provides contextually relevant responses with low latency.

Though GitHub's always experimenting with new models as they come out, Codex was the first really powerful generative AI model that was available, said David Slater,

GitHub Copilot
How GitHub builds GitHub
LLM

More on AI

Scaling AI opportunity across the globe: Learnings from GitHub and Andela

Developers connected to Andela share how they're learning AI tools inside real production workflows.

Admas Kanyagia & Ali Condah

Welcome to the Eternal September of open source. Here's what we plan to do for maintainers.

Open source is hitting an "Eternal September." As contribution friction drops, maintainers are adapting with new trust signals, triage approaches, and community-led solutions.

Ashley Wolf

improvements was invaluable.

Inside GitHub: Working with the LLMs behind GitHub Copilot

Get the full, inside story from researchers and engineers at GitHub who built the early versions of GitHub Copilot and have improved it since then.

All that experimentation resulted in a pair programmer that, ultimately, frees up a developer's time to focus on more fulfilling work. The tool is often a huge help even for starting new projects or files from scratch because it scaffolds a starting point that developers can adapt and tweak as desired, said Alice Li, a ML researcher at GitHub.

> "
>
> I still find myself impressed and even surprised by what GitHub Copilot can do, even after having worked on it for some time now.
>
> – Alice Li, ML researcher at GitHub

Why context matters

code. When it comes to a generative AI coding tool, we need to teach that tool what information to use to do the same.

Transformer LLMs are good at connecting the dots and big-picture thinking. Generative AI coding tools are made possible by large language models (LLMs). These models are sets of algorithms trained on large amounts of code and human language. Today's state-of-the-art LLMs are transformers, which makes them adept at making connections between text in a user's input and the output that the model has already generated. This is why today's generative AI tools are providing responses that are more contextually relevant than previous AI models.

But they need to be told what information is relevant to your code. Right now, transformers that are fast enough to power GitHub Copilot can process about 6,000 characters at a time. While that's been enough to advance and accelerate tasks like code completion and code change summarization, the limited amount of characters means that not all of a developer's code can be used as context.

So, our challenge is to figure out not only what data to feed the model, but also how to best order and enter it to get the best suggestions for the developer.

Learn more about LLMs, generative AI coding tools, and how they're changing the way developers work.

It all comes down to prompts, which are compilations of IDE code and relevant context that's fed to the model. Prompts are generated by algorithms in the background, at any point in your coding. That's why GitHub Copilot will generate coding suggestions whether you're currently writing or just finished a comment, or in the middle of some gnarly code.

- **Here's how a prompt is created**: a series of algorithms first select relevant code snippets or comments from your current file and other sources (which we'll dive into below). These snippets and comments are then prioritized, filtered, and assembled into the final prompt.

GitHub Copilot's contextual understanding has continuously matured over time. The first version was only able to consider the file you were working on in your IDE to be contextually relevant. But we knew context went beyond that. Now, just a year later, we're experimenting with algorithms that will consider your entire codebase to generate customized suggestions.

Let's look at how we got here:

- **Prompt engineering** is the delicate art of creating a prompt so that the model makes the most useful prediction for the user. The prompt tells LLMs, including GitHub Copilot, what data, and in what order, to process in order to contextualize your code. Most of this work takes place in what's called a **prompt library**, which is where our in-house ML experts work with algorithms to extract and prioritize a variety of sources of information about the

- **Neighboring tabs** is what we call the technique that allows GitHub Copilot to process all of the files open in a developer's IDE instead of just the single one the developer is working on. By opening all files relevant to their project, developers automatically invoke GitHub Copilot to comb through all of the data and find matching pieces of code between their open files and the code around their cursor—and add those matches to the prompt.

When developing neighboring tabs, the GitHub Next team and in-house ML researchers did A/B tests to figure out the best parameters for identifying matches between code in your IDE and code in your open tabs. They found that **setting a very low bar for when to include a match actually made for the best coding suggestions**.

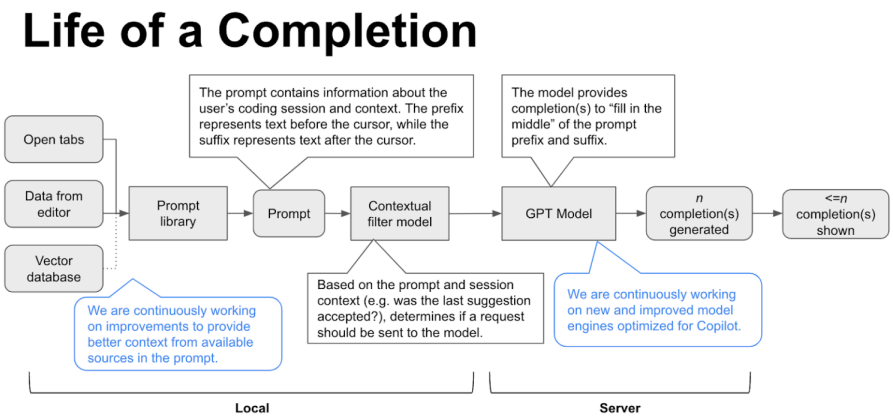
By including every little bit of context, neighboring tabs helped to relatively increase user acceptance of GitHub Copilot's suggestions by 5%**.

> "
>
> Even if there was no perfect match—or even a very good one—picking the best match we found and including that as context for the model was better than including nothing at all.
>
> – Albert Ziegler, principal ML engineer at GitHub

code before your cursor would be put into the prompt –ignoring the code after your cursor. (At GitHub, we refer to code before the cursor as the prefix and after the cursor as the suffix.) **With FIM, we can tell the model which part of the prompt is the prefix, and which part is the suffix**.

Even if you're creating something from scratch and have a skeleton of a file, we know that coding isn't linear or sequential. So, while you bounce around your file, FIM helps GitHub Copilot offer better coding suggestions for the part in your file where your cursor is located, or the code that's supposed to come between the prefix and suffix.

Based on A/B testing, **FIM gave a 10% relative boost in performance, meaning developers accepted 10% more of the completions that were shown to them**. And thanks to optimal use of caching, neighboring tabs and FIM work in the background without any added latency.



Simplified system diagram focused on model quality efforts. Made by Alice Li, machine learning researcher at GitHub.

Improving semantic understanding

developers working in private repositories or with proprietary code. Generative AI coding tools use something called embeddings to retrieve information from a vector database.

- **What's a vector database?** It's a database that indexes high-dimensional vectors.
- **What's a high-dimensional vector?** They're mathematical representations of objects, and because these vectors can model objects in a number of dimensions, they can capture complexities of that object. When used properly to represent pieces of code, they may represent both the semantics and even intention of the code—not just the syntax.
- **What's an embedding?** In the context of coding and LLMs, an embedding is the representation of a piece of code as a high-dimensional vector. Because of the "knowledge" the LLM has of both programming and natural language, it's able to capture both the syntax and semantics of the code in the vector.

Here's how they'd all work together:

- Algorithms would create embeddings for all snippets in the repository (potentially billions of them), and keep them stored in the vector database.
- Then, as you're coding, algorithms would embed the snippets in your IDE.
- Algorithms would then make approximate matches—also, in real time—between the embeddings that are created for your IDE snippets and the embeddings already stored in the vector database. The vector database is what allows algorithms to quickly search for approximate matches (not just exact ones) on the

Developers are familiar with retrieving data with hashcodes, which typically look for exact character by character matches, explained Alireza Goudarzi, senior ML researcher at GitHub. “But embeddings—because they arise from LLMs that were trained on a vast amount of data—develop a sense of semantic closeness between code snippets and natural language prompts.”

Read the three sentences below and identify which two are the most semantically similar.

- **Sentence A**: The king moved and captured the pawn.
- **Sentence B**: The king was crowned in Westminster Abbey.
- **Sentence C**: Both white rooks were still in the game.

The answer is sentences A and C because both are about chess. While sentences A and B are syntactically, or structurally similar because both have a king as the subject, they’re semantically different because “king” is used in different contexts.

Here’s how each of those statements could translate to Python. Note the syntactic similarity between snippets A and B despite their semantic difference, and the semantic similarity between snippets A and C despite their syntactic difference.

Snippet A:

```
if king.location() == pawn.location():
    board.captures_piece(king, pawn)
```

Snippet B:

Snippet C:

```
if len([ r for r in board.pieces("white") if r.typ
    return True
```

As mentioned above, we're still experimenting with retrieval algorithms. We're designing the feature with enterprise customers in mind, specifically those who are looking for a customized coding experience with private repositories and would explicitly opt in to use the feature.

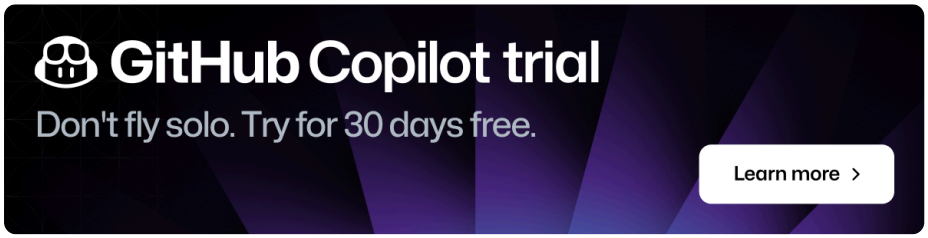
Take this with you

Last year, we conducted quantitative research on GitHub Copilot and found that developers code up to 55% faster while using the pair programmer. This means developers feel more productive, complete repetitive tasks more quickly, and can focus more on satisfying work. But our work won't stop there.

The GitHub product and R&D teams, including GitHub Next, have been collaborating with Microsoft Azure AI-Platform to continue bringing improvements to GitHub Copilot's contextual understanding. So much of the work that helps GitHub Copilot contextualize your code happens behind the scenes. While you write and edit your code, GitHub Copilot is responding to your writing and edits in real time by generating prompts–or, in other words, prioritizing and sending relevant information to the model based on your actions in your IDE—to keep giving you the best coding suggestions.

powered software development. Discover what's new.

- Learn how the LLMs powering GitHub Copilot are getting better.
- Read our research on how GitHub Copilot is impacting developer productivity.



Tags: AI | AI Insights | generative AI | GitHub Copilot | How GitHub builds GitHub | LLM

Written by

Johan Rosenkilde
@johanrosenkilde

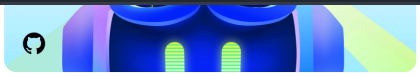
Related posts

AI & ML

Continuous AI for accessibility: How GitHub transforms feedback into inclusion

AI automates triage for accessibility feedback, allowing us to focus on fixing barriers—turning a chaotic backlog into continuous, rapid resolutions.

Carie Fisher



AI & ML

The era of "AI as text" is over. Execution is the new interface.

AI is shifting from prompt-response interactions to programmable execution. See how the GitHub Copilot SDK enables agentic workflows directly inside your applications.

Gwen Davis

AI & ML

Under the hood: Security architecture of GitHub Agentic Workflows

GitHub Agentic Workflows are built with isolation, constrained outputs, and comprehensive logging. Learn how our threat model and security architecture help teams run agents safely in GitHub Actions.

Landon Cox & Jiaxiao Zhou

Explore more from GitHub

Docs

Everything you need to master GitHub, all in one place.

Go to Docs

GitHub

Build what's next on GitHub, the place for anyone from anywhere to build anything.

Start building

Customer stories

The GitHub Podcast

Learn more

open source developer community on GitHub.

Listen now

We do newsletters, too

Discover tips, technical guides, and best practices in our biweekly newsletter just for devs.

Your email address

* Your email address

Subscribe >

Yes please, I'd like GitHub and affiliates to use my information for personalized communications, targeted advertising and campaign effectiveness. See the GitHub Privacy Statement for more details.

© 2026 GitHub, Inc. Terms Privacy Manage Cookies
Do not share my personal information